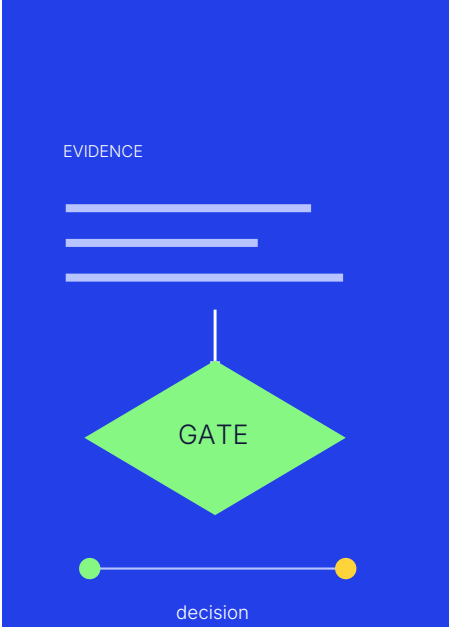
LECTURE 07 / WEEK 04

MLOps: Machine Learning Operations

Automated Training Pipelines with Evaluation Gates

How a training run produces evidence, earns eligibility and reaches a service through a controlled decision.



Three questions organize the lecture

A pipeline makes a claim inspectable; a gate makes a decision explicit.

1 **Reproducibility**

Which data, code, configuration and environment produced this artifact?

2 **Evaluation**

Is the comparison valid, and do all declared criteria pass?

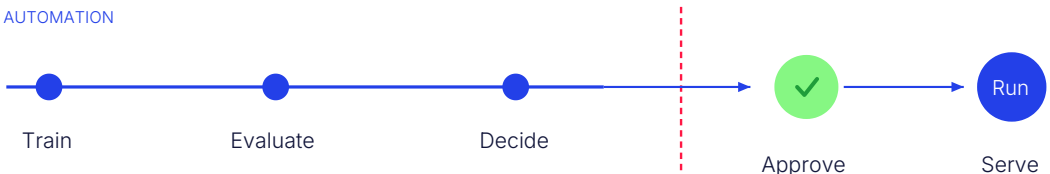
3 **Authority**

Who may change the served version, and how is the change verified?

A successful computation is evidence about execution, not permission to deploy.

Training success creates a candidate

Execution, quality and release are different decisions.



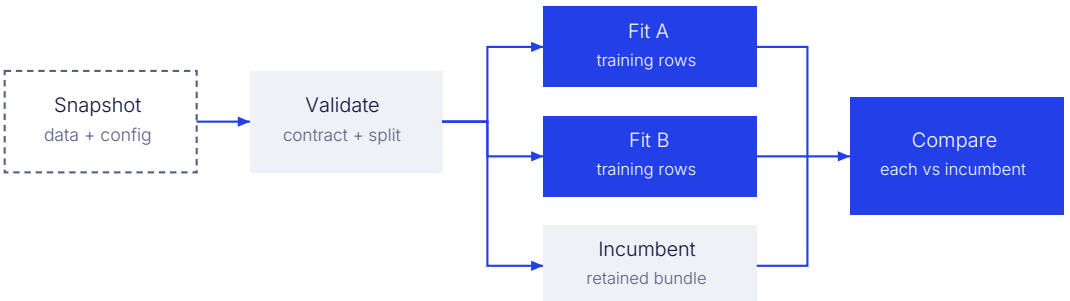
A completed job supplies evidence. The gate applies a policy. Release authority remains explicit.

The approved identity must match the running identity.

Automatic training can coexist with reviewed manual promotion.

A DAG encodes dependencies

Directed edges say which outputs must exist before another step can start.

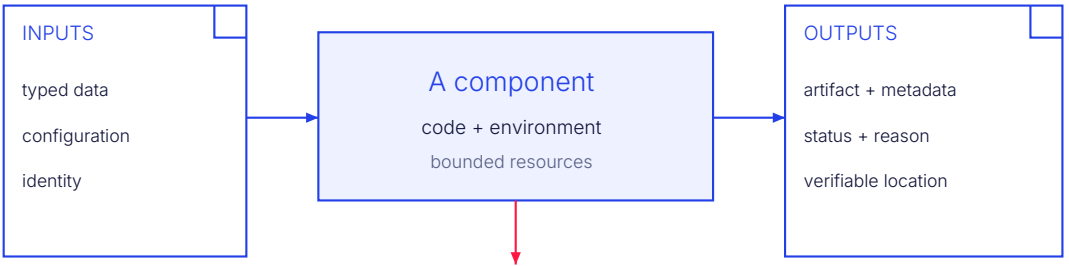


Same validation rows + fixed policy

A workflow can branch and join; it need not be one long sequential script.

Each stage has an explicit contract

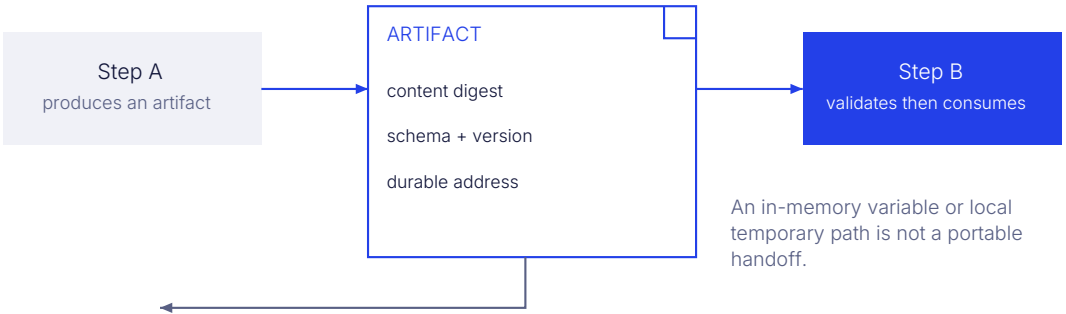
A component is a unit of work, not necessarily a separate container.



Process boundaries follow operational needs; separation into functions is a valid starting point.

Pass references with their evidence

Downstream work must know which bytes it consumed.



An in-memory variable or local temporary path is not a portable handoff.

Artifacts outlive a worker process

A path alone is a location; a digest identifies the bytes found there.

Reproducibility needs more than a seed

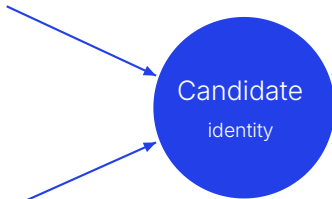
Record the inputs that can change the computation.

Data

snapshot + row boundary

Config

features + parameters



Code

revision + dependencies

Run

seed + execution record

An identifier is useful when its evidence can be retrieved.

Repeatable predictions do not imply byte-identical model files on every system.

Retry computation; preserve meaning

Retrying a transient failure must not duplicate side effects.

Transient execution fault



bounded retry

Invalid evidence

repair, then rerun

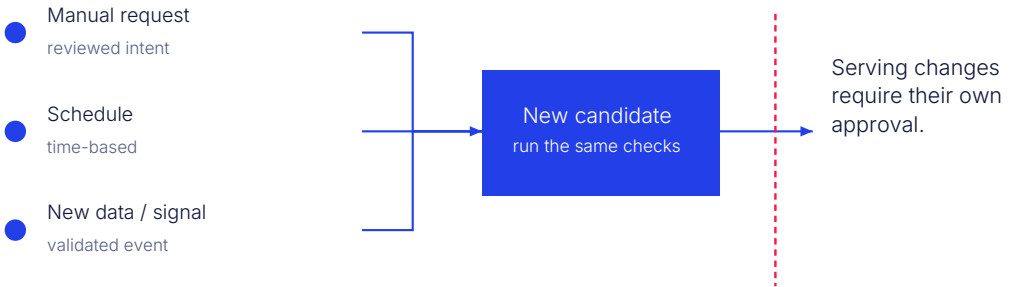
Valid evidence fails policy

record and stop

Idempotence concerns the effect of a replay; it is different from numerical determinism.

A trigger starts work; it grants no approval

Manual, scheduled and data-driven runs need the same evaluation boundary.

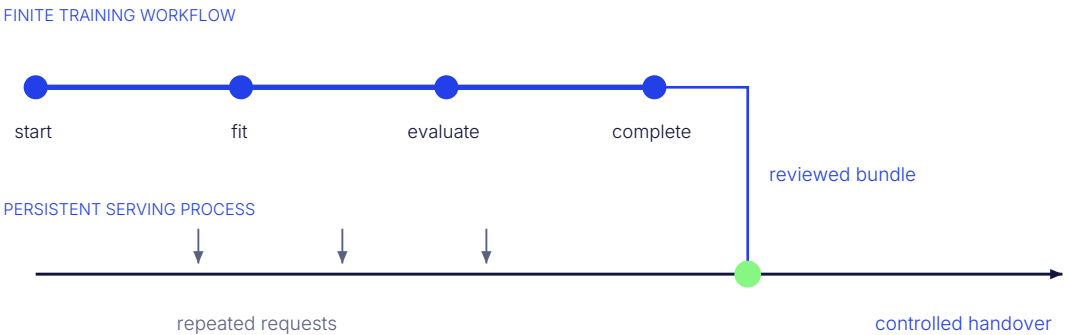


A trigger starts computation. It supplies no exemption from evaluation or review.

Trigger frequency and training budget are policy choices.

Training and serving have different lifetimes

A finite workflow produces a bundle; a service repeatedly answers requests.



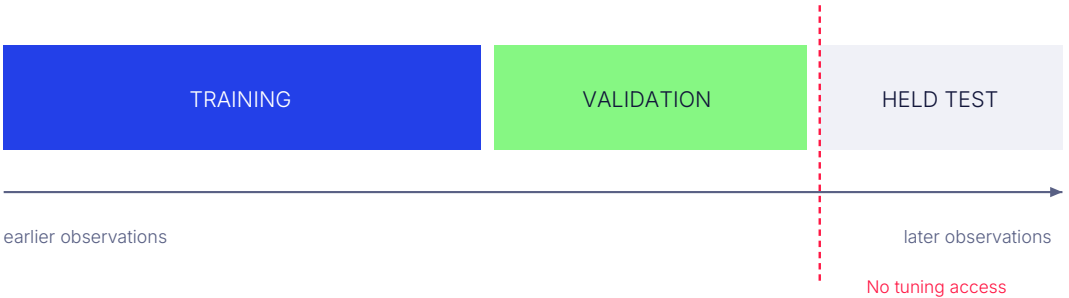
A training component may call a service; the two abstractions still have different responsibilities.

Protect the future from model selection

For temporal data, the direction of information matters.

Fit the model and learned preprocessing.

Select using validation.



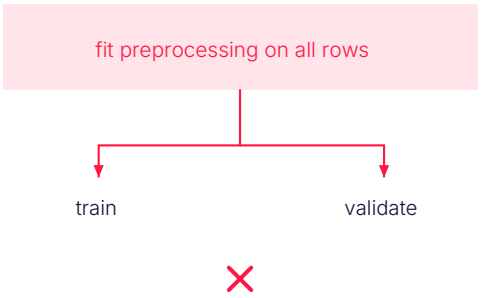
Use the project's declared boundaries. The final test evaluates an already selected model.

The chronology is a protocol, not merely three variable names.

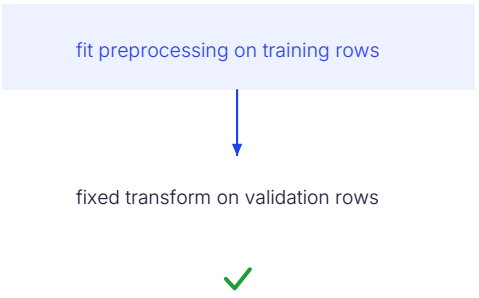
Preprocessing belongs inside the boundary

Any learned transformation must be fitted only on the allowed training rows.

LEAKING PROCEDURE



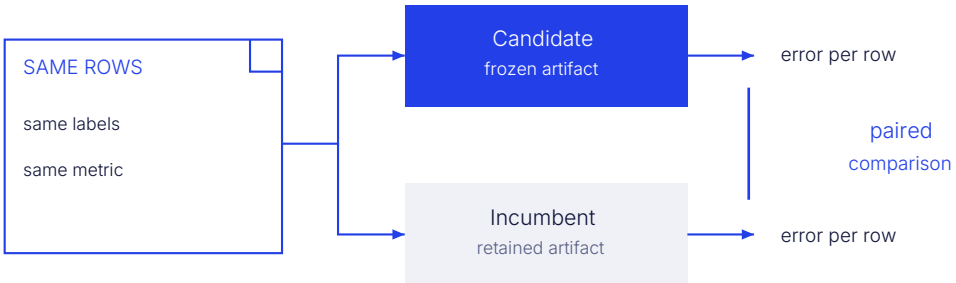
BOUNDARY PRESERVED



A scikit-learn Pipeline helps enforce fit/transform order; feature availability still needs a design review.

Compare predictions on the same observations

A candidate cannot be fairly compared with an unexplained historical score.

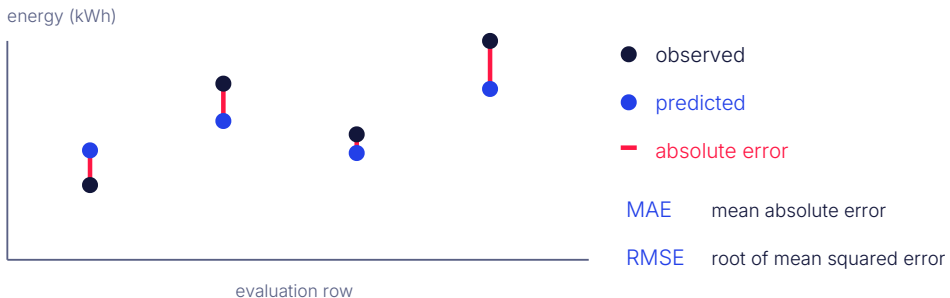


A metric from another data window, target, or feature contract is not a fair incumbent comparison.

A matching metric name alone does not establish a fair comparison.

A metric defines what error means

Illustrative hourly building-energy regression; the unit is kWh.

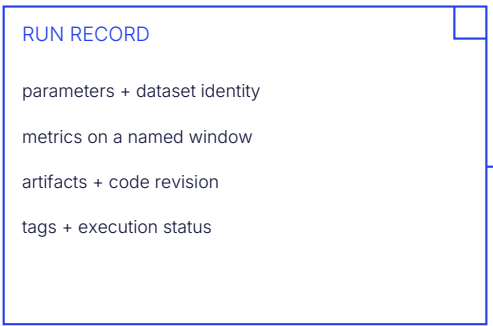


Synthetic geometry. Both metrics use kWh; squaring gives large residuals more influence.

Choose the primary metric and aggregation before evaluating candidates.

Tracking records how a model was produced

An experiment groups runs; a run joins choices, measurements and artifacts.



QUESTIONS A RUN CAN ANSWER

What ran?

With which inputs?

Where is the output?

A tracking record does not itself grant release approval.

A local MLflow database is local evidence until another environment can actually retrieve it.

Tracking, registry and serving are distinct

Each answers a different operational question.

TRACKING

What happened in the experiment?

REGISTRY / ACCEPTED STORE

Which versions are approved and recoverable?

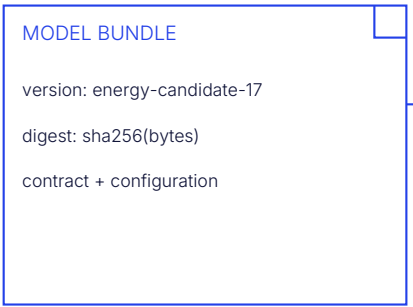
SERVING

Which version is answering now?

Changing an MLflow alias does not by itself reload a separately running FastAPI process.

Bind the decision to exact artifact bytes

A model name, a run ID and a file hash answer different questions.



Version identifier

Names the trained instance for its lifetime.

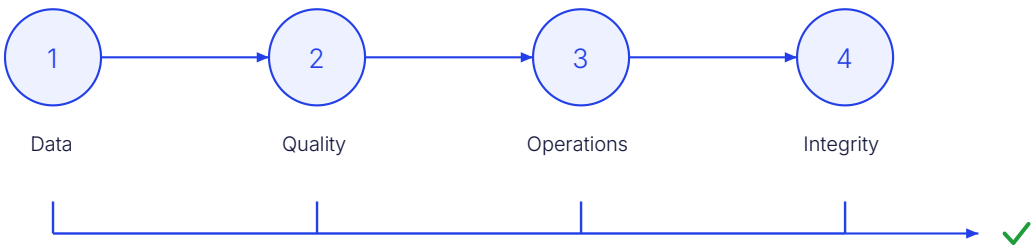
Content digest

Detects byte changes against a trusted expected record.

Verify a trusted record before loading a pickle-based artifact.

A gate is a policy over evidence

All required criteria must pass before a candidate is eligible.



All required evidence

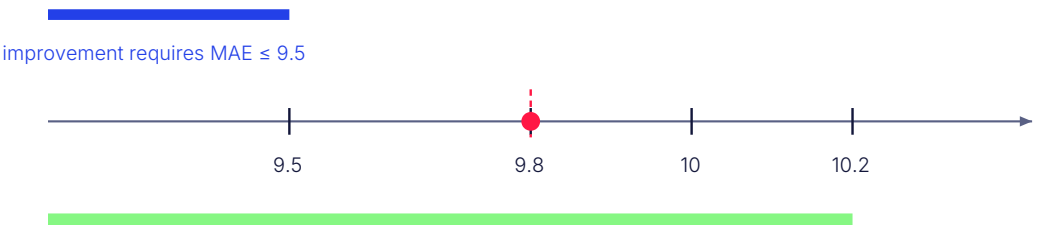
Missing / invalid = no promotion

Commit the policy before candidate scores; rule changes need review. Missing required evidence never passes.

Two valid policies make different choices

Illustrative energy model: incumbent MAE 10.0 kWh; candidate MAE 9.8 kWh.

LOWER VALIDATION MAE IS BETTER



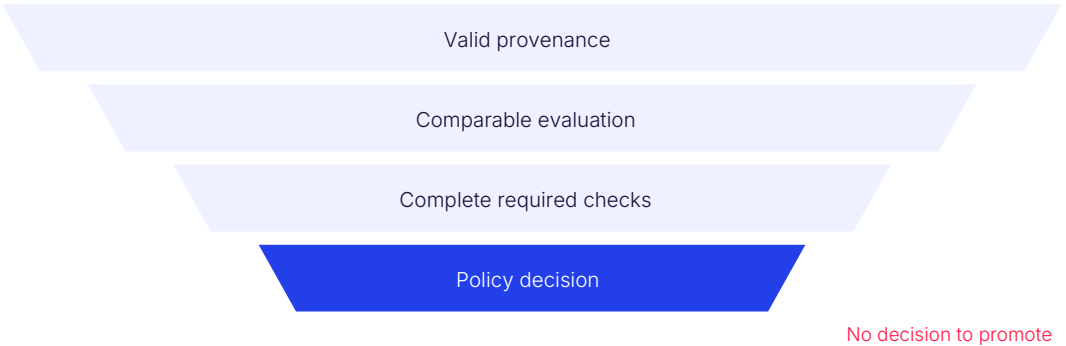
non-regression allows MAE ≤ 10.2

candidate 9.8: fails improvement; passes non-regression

These numbers illustrate policy semantics; they are not recommended thresholds.

Validate evidence before comparing numbers

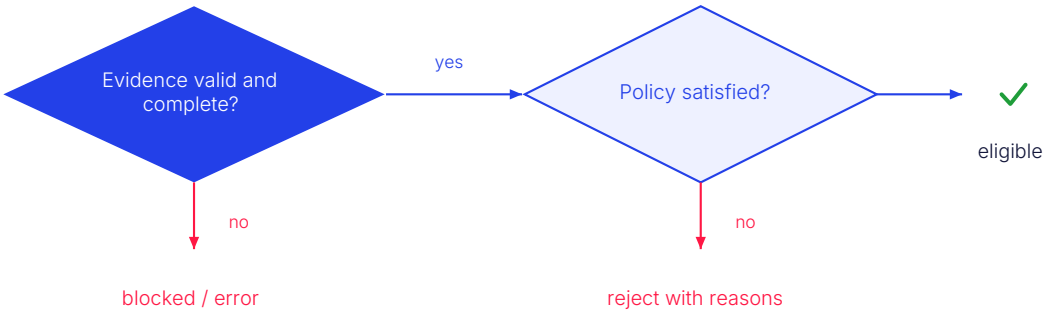
An absent or invalid measurement must never become an implicit pass.



Treat invalid or unavailable required evidence as blocked eligibility, with a reason.

A blocked gate is different from a reject

Both prevent promotion, but they require different responses.



A record preserves the outcome and reasons; an infrastructure retry is not a new policy.

Reject invalid numbers explicitly

A defensive primitive, not a complete quality gate.

ILLUSTRATIVE PYTHON

For JSON-style numbers, reject booleans, non-finite values and integers too large for float conversion before applying inequalities.

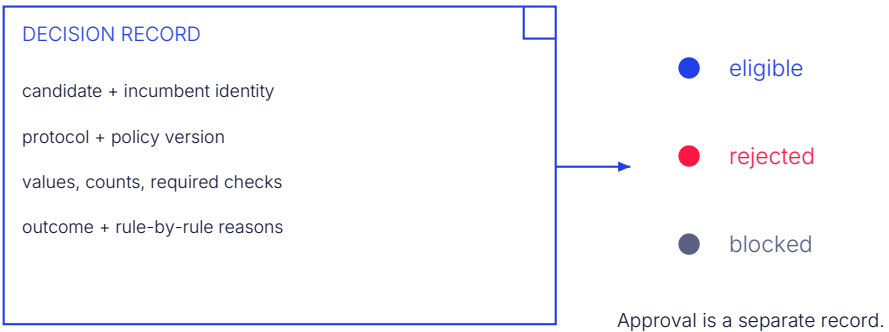
STANDALONE FUNCTION

```
from math import isfinite
def finite_number(value):
    try:
        return (type(value) in (int, float)
                and isfinite(value))
    except OverflowError:
        return False
```

The complete gate must also validate provenance, counts, required checks and artifact identity.

A decision record explains the decision

The record must allow a reviewer to reconstruct both policy and evidence.



Approval is a separate record.

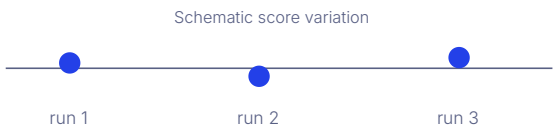
Record facts the evaluator observed; do not read back a self-authored passed flag as proof.

Two uncertainties need different evidence

Numerical repeatability is not uncertainty about future performance.

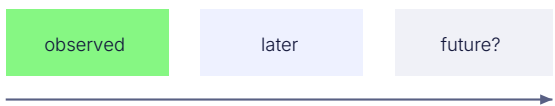
EXECUTION VARIATION

Hold data and protocol fixed; repeat execution.



SAMPLING / FUTURE CONDITIONS

One validation period cannot represent every future condition.



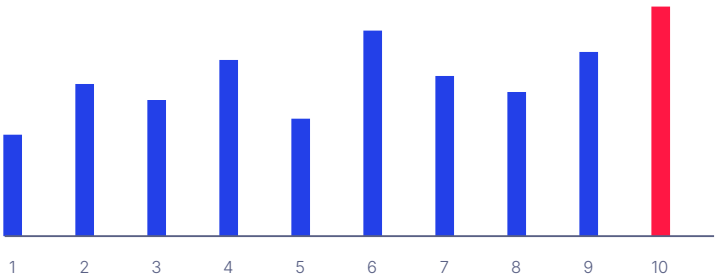
Choose an application-relevant margin before interpreting a small gain.

Repeating the same deterministic run does not create new independent observations.

Repeated selection consumes validation evidence

The best of many noisy results tends to look better than it is.

ONE VALIDATION WINDOW, MANY CHOICES



The selected winner benefited from repeated opportunities to look good.

Retain a separate final test.

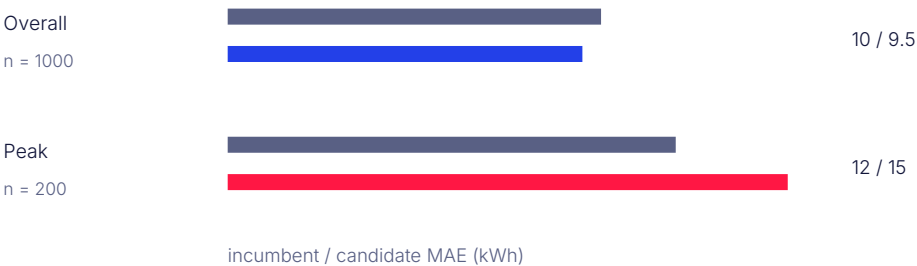
Schematic apparent improvement across candidates. Repeated tuning can overfit validation.

A gate standardizes a decision; it does not remove overfitting to the validation period.

An aggregate can hide a regression

A better overall score may conceal harm in a meaningful slice.

SAME VALIDATION COHORT

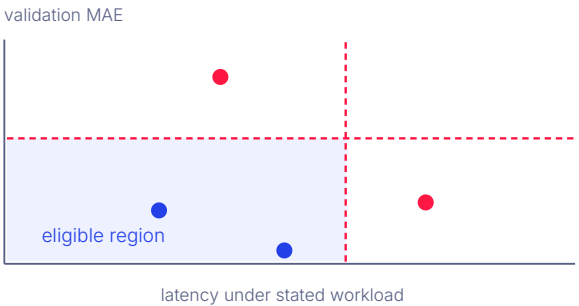


Synthetic values

Illustrative energy example: lower overall MAE, but worse error during peak demand.

Operational bounds are part of eligibility

A model must fit the environment that will actually serve it.



Quality and serving constraints apply together.

Measure on the intended runtime. Include integrity and contract checks.

Schematic limits. Eligibility does not replace release approval.

Unavailable serving evidence cannot be represented as passed.

Measure the path that users experience

Model predict time is only one part of request latency.



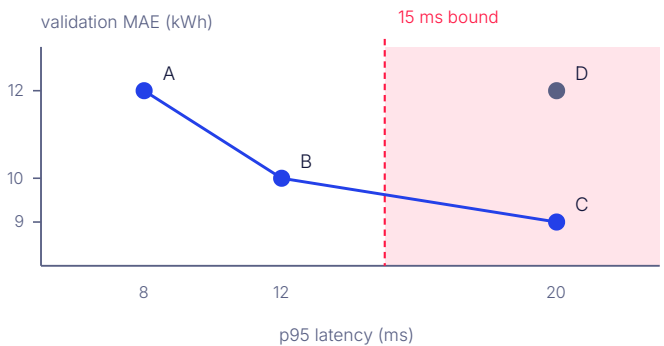
28 ms total for this request

Synthetic example

A small warm microbenchmark is useful evidence, but it does not establish a service SLO.

Optimization changes a trade-off

Choose among candidates that satisfy hard constraints.



SYNTHETIC DATA

A and B meet the latency bound. C has lower MAE but exceeds the bound.

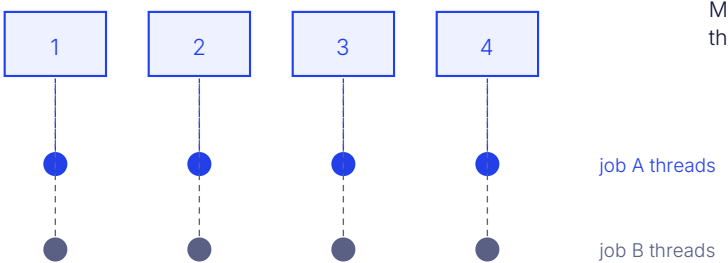
D is dominated by B.

A smaller or faster model is not automatically less accurate; measure the result.

Bound compute before increasing it

More workers can increase overhead, memory pressure and contention.

ILLUSTRATIVE: FOUR AVAILABLE CORES



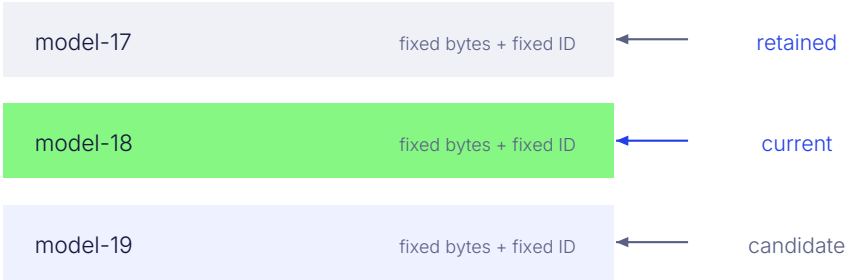
More workers can contend for the same cores and memory.

Profile wall time, memory and quality. Nested parallelism can oversubscribe the machine.

An n_jobs setting controls supported CPU parallelism; it does not move scikit-learn training onto a GPU.

Keep versions immutable; move references

Approval changes status, not the identity of the trained artifact.

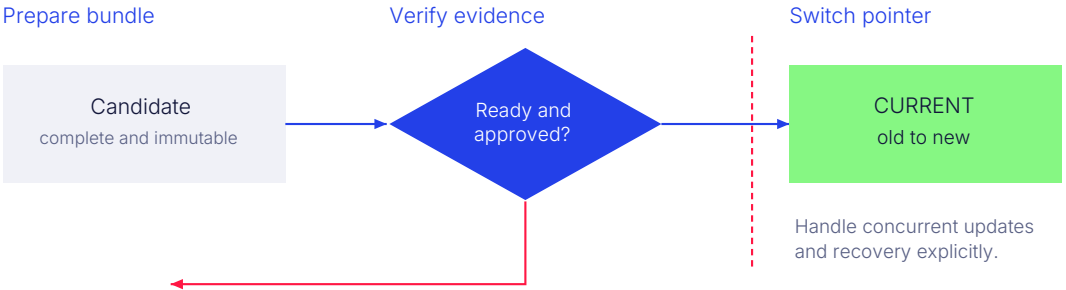


Move the status or pointer. Do not rename or overwrite the identity of a trained model.

MLflow aliases are mutable references; record the concrete version they resolved to.

Promotion is a controlled transaction

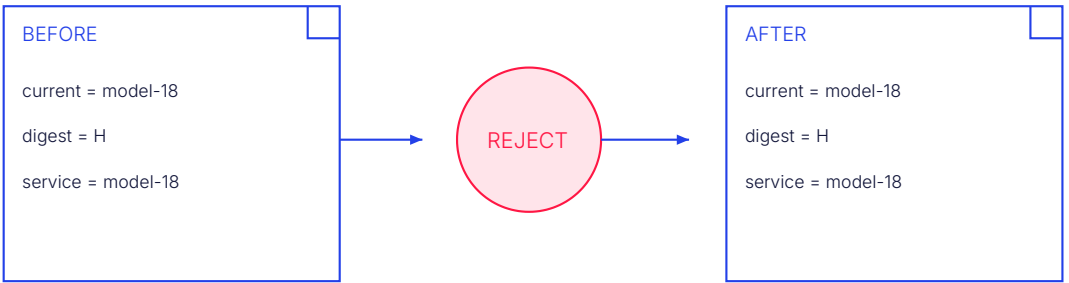
Approve the exact candidate and preserve the version it replaces.



A local atomic rename is not a transaction across a registry, database and service.

Rejection must preserve the accepted state

A rejected candidate may leave evidence, but it must not change the serving selection.

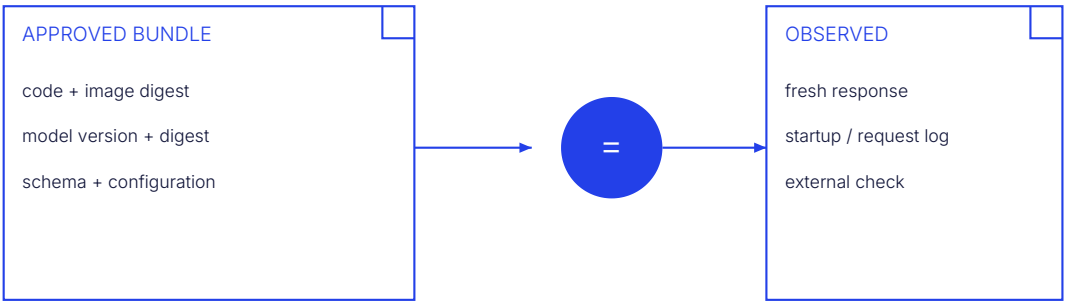


The decision log changes. The accepted bundle does not.

A rejection is a useful tested outcome, not an unsuccessful experiment to hide.

Verify what the process actually loaded

A successful promotion record is not yet proof of a healthy deployment.



Verify the identity actually serving requests, not only the files present on disk.

Restart or reload behavior is explicit; rollback should not require retraining.

Automated gates have a limited claim

The conclusion is conditional on evidence, protocol and context.



Still outside this claim

Future conditions

Missing requirements

Unmeasured failure modes

Review + monitoring + recovery

A gate is an explicit acceptance policy, not a proof that the world cannot change.

ENGINEERING PRINCIPLE

Train automatically.
Decide from evidence.
Promote deliberately.

Preserve the evaluation boundary, the exact artifact and the version you may need to recover.

OPTIONAL THEORY / NOT PROJECT REQUIREMENTS

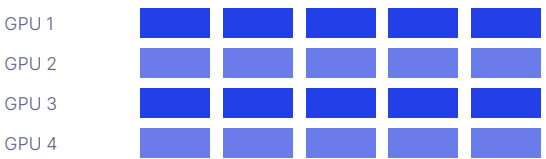
Scale only after measuring the constraint.

Compute, parallel training, compression, serving runtimes and managed pipelines. The course project keeps its existing stack.

Work, rate and elapsed time are different

Optional deep dive: Training cost needs units and assumptions.

work		achieved rate		elapsed time
FLOP	=	FLOP / s	×	s



5 days on each of 4 GPUs

20 GPU-days

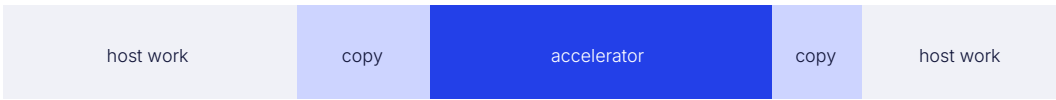
Device-time, not a hardware-independent amount of work.

Illustrative: 4 GPUs × 5 days = 20 GPU-days; different GPUs need not do equal work.

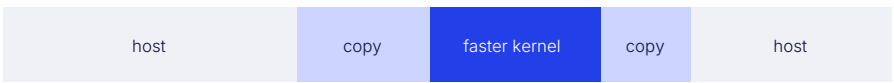
Match hardware to the computation

Optional deep dive: An accelerator helps only when the workload can use it.

THE WHOLE REQUEST



A FASTER KERNEL CHANGES ONE SEGMENT

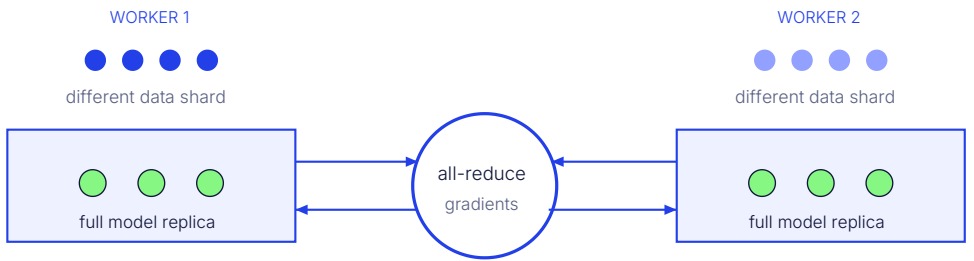


schematic; no measured speedup

More compute capacity does not guarantee faster training, lower cost or better quality.

Data parallelism replicates the model

Optional deep dive: Workers process different data while synchronizing learning.



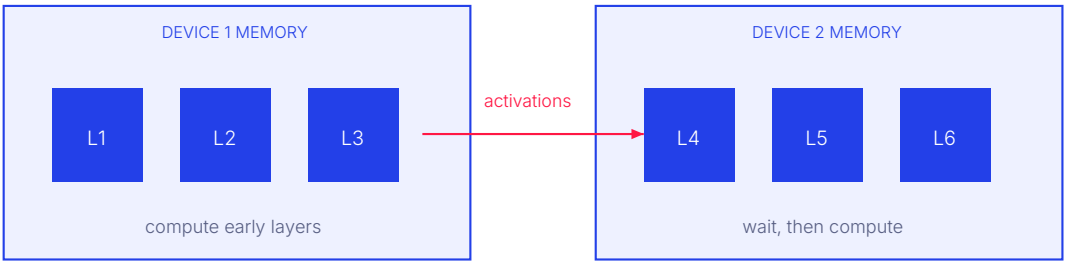
each replica applies a consistent update

communication and changed batch size affect the outcome

N workers do not imply N-times faster convergence. The model generally must fit each replica's device.

Model parallelism splits the model

Optional deep dive: Partitioning helps a model fit, but moves intermediate values.

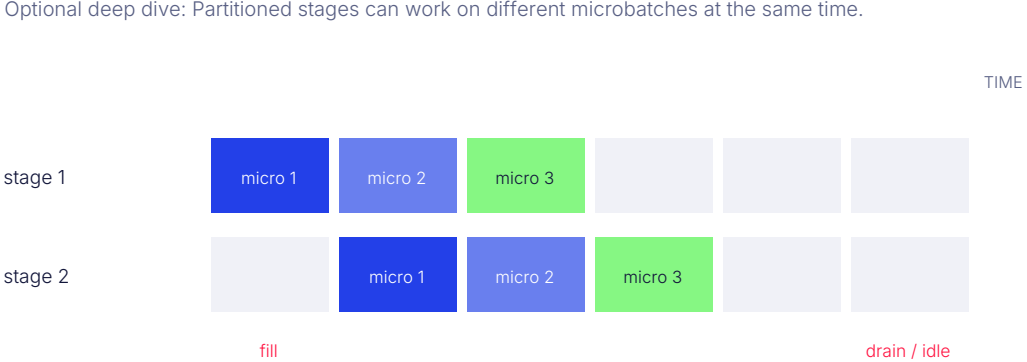


Tensor parallelism instead shards an operation or its parameters.

Combined memory is not a single free shared memory pool; the partition must respect dependencies.

Pipeline parallelism overlaps microbatches

Optional deep dive: Partitioned stages can work on different microbatches at the same time.

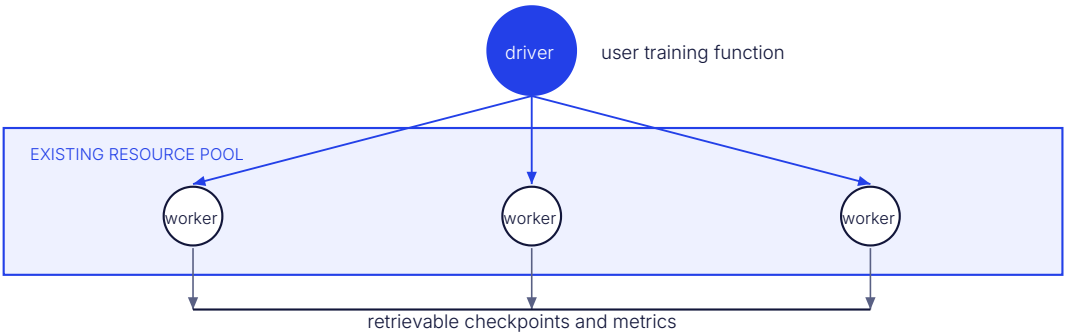


forward-only schematic; real training also schedules backward passes

Pipeline parallelism is a training execution technique; a workflow DAG is a different abstraction.

An execution framework manages workers

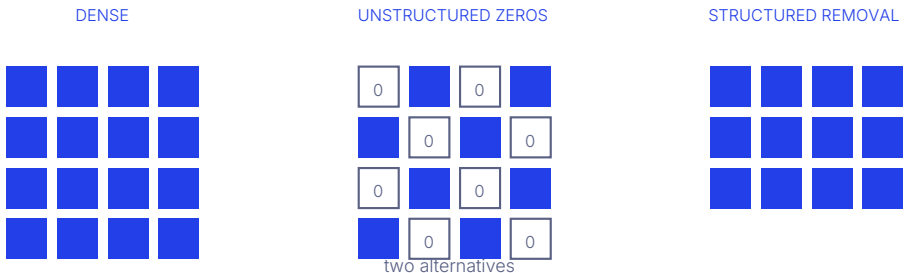
Optional deep dive: Ray is an example of software coordinating training resources.



Distributed execution adds operational cost; use it for a measured capacity or throughput need.

Pruning changes the representation

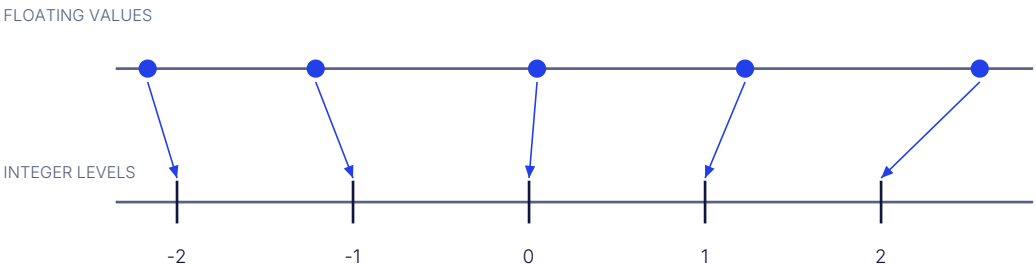
Optional deep dive: Removing weights or structure may reduce storage or computation.



Sparsity is not a speed guarantee; encoding, kernel support and hardware decide the benefit.

Quantization trades precision for efficiency

Optional deep dive: The quantized artifact is a new candidate that must be evaluated.

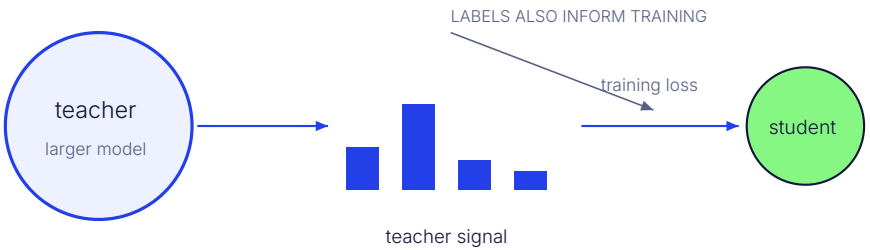


schematic levels; actual scale, zero point and range are chosen

Lower precision can reduce storage; speed and accuracy depend on operators, calibration and hardware.

Distillation trains a different model

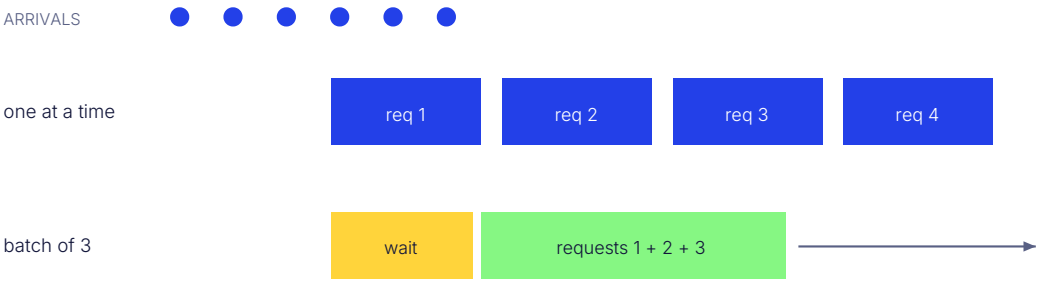
Optional deep dive: A student learns from a teacher's outputs or representations.



Softened class probabilities are one method; distillation is broader than high-temperature softmax.

Scaling and batching change queueing

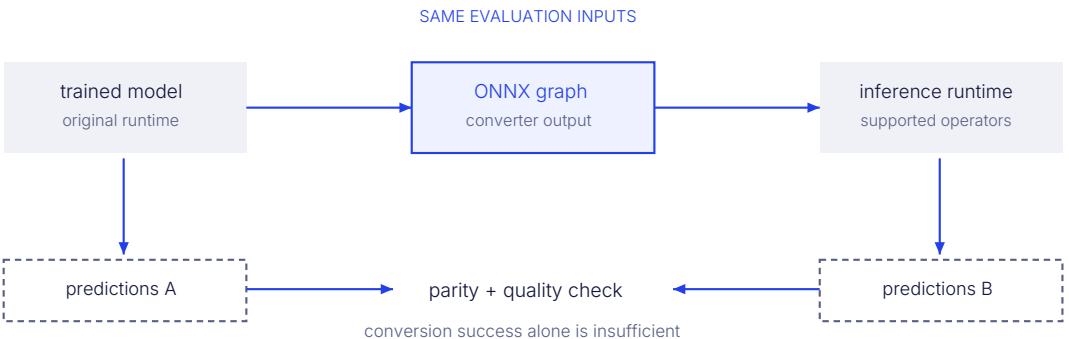
Optional deep dive: Throughput and latency respond to different bottlenecks.



Autoscaling can change request latency through queueing; it is not guaranteed to leave it unchanged.

ONNX is a format, not a performance promise

Optional deep dive: Conversion separates a model representation from its training framework.

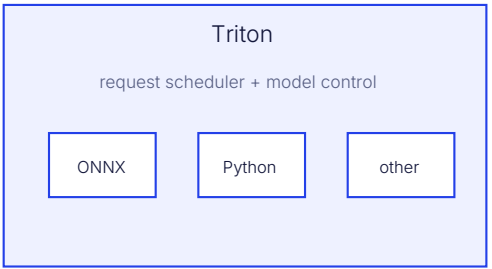


Not every estimator or operator converts; a converted model must pass quality and service checks.

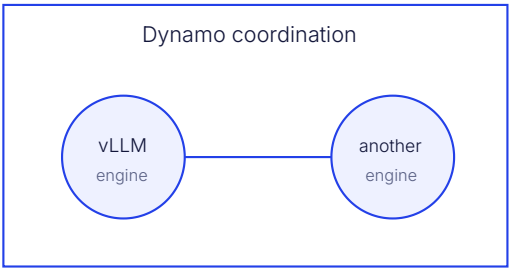
Serving frameworks solve different problems

Optional deep dive: Choose from a measured workload and maintenance needs.

MULTI-BACKEND SERVER



DISTRIBUTED GENERATIVE SERVING

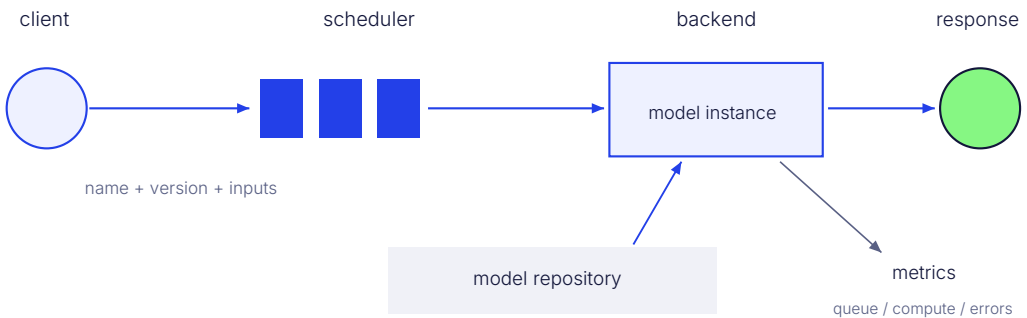


roles can compose; no universal speed ranking

TorchServe appears in older material; its official documentation now states limited maintenance.

A server schedules model execution

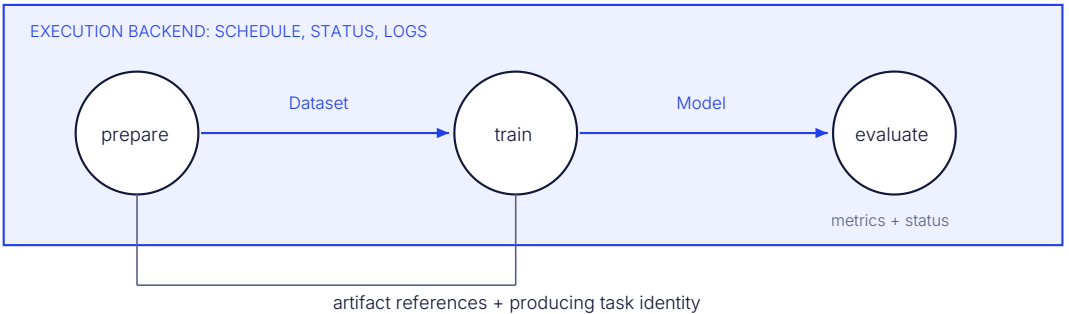
Optional deep dive: Triton illustrates the boundary between request handling and execution.



Model instances are not the same as automatic scaling of the surrounding cluster.

Managed components keep explicit interfaces

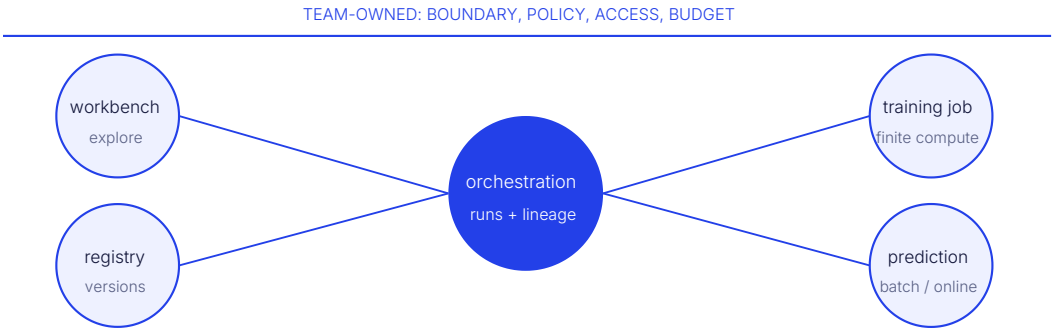
Optional deep dive: A platform makes the same dependency contract executable at larger scope.



Kubeflow components may share base images; one component does not require one unique image.

Platforms package several responsibilities

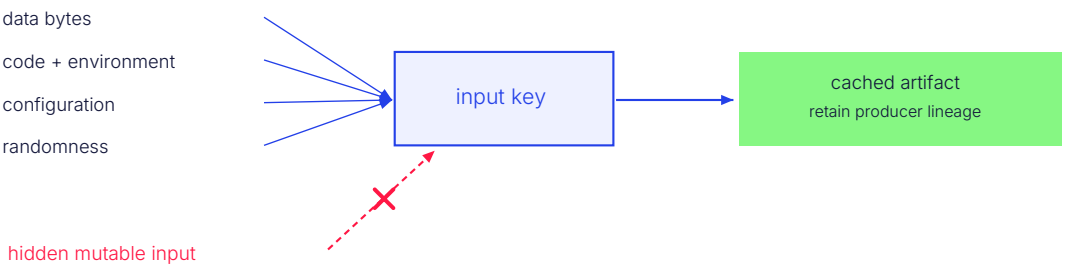
Optional deep dive: SageMaker, Vertex AI and Azure ML each package these roles.



The team still owns data boundaries, gate policy, permissions, costs and verification.

A cache is valid only for its true inputs

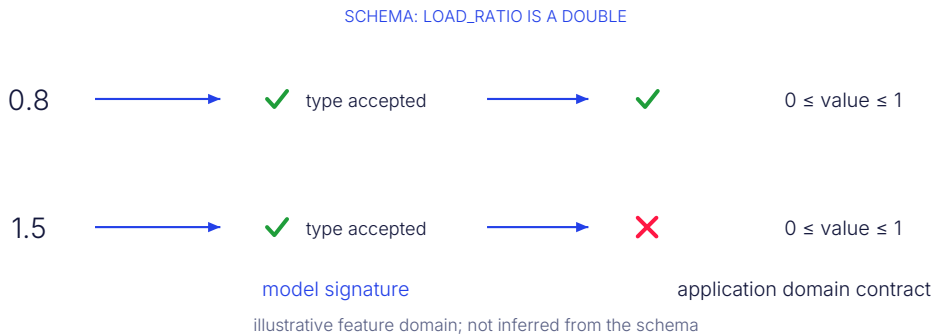
Optional deep dive: Reusing work is correct only when the cached result still answers the same computation.



A cache hit is evidence of reuse, not proof of a fresh execution.

Signatures describe format, not validity

Optional deep dive: MLflow model signatures add a schema to a logged model.

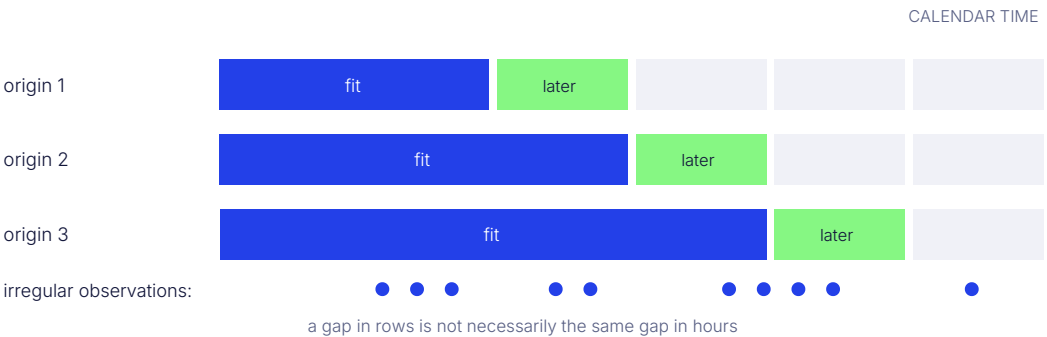


illustrative feature domain; not inferred from the schema

Logging a raw joblib file as an artifact does not automatically create an MLflow model signature.

Temporal validation needs calendar logic

Optional deep dive: Rolling-origin analysis can expose period sensitivity before final evaluation.

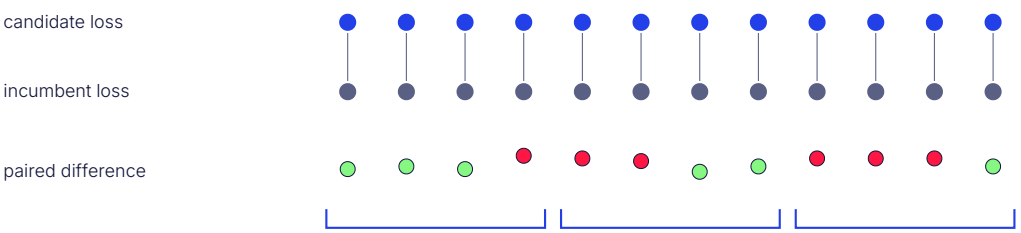


TimeSeriesSplit counts rows; its equal-duration comparison assumes equally spaced samples.

Compare uncertainty on paired observations

Optional deep dive: Uncertainty analysis must respect dependence and the selection process.

SAME OBSERVATIONS

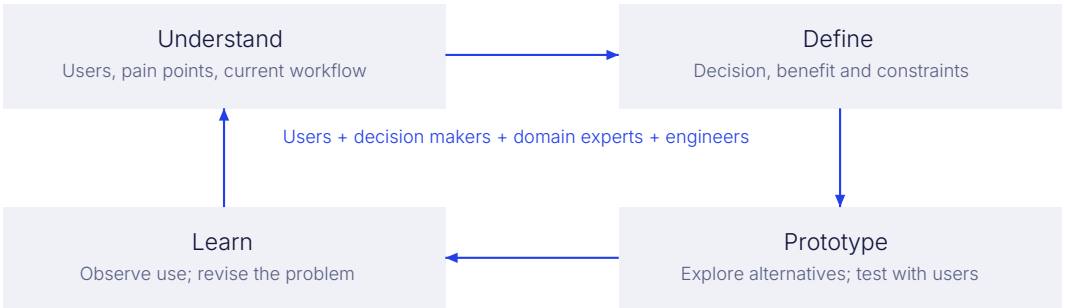


preserve time blocks; schematic losses, no interval estimate

An interval under chosen assumptions is not a guarantee about every future operating regime.

Discover the problem before the model

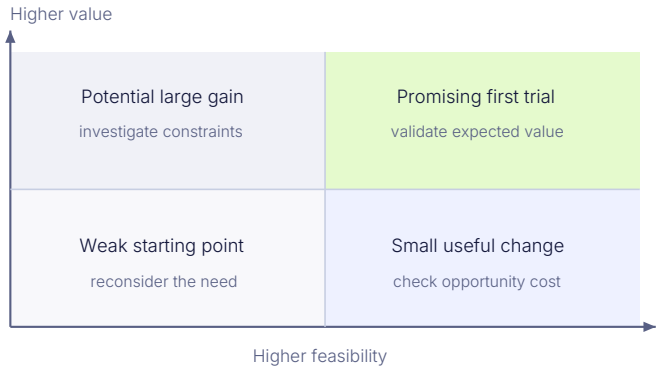
A facilitated learning cycle connects a use case to a real workflow.



Prototype and test the idea early; revisit assumptions as evidence changes.

Prioritize value and feasibility together

A qualitative matrix makes assumptions visible; it does not prove a return.



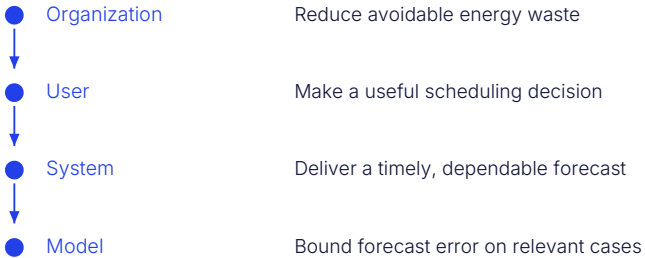
Compare evidence about data, delivery effort, risk and user benefit.

Positions are judgments to revisit, not measured scores.

Choose an initial scope, then revise the roadmap using evidence from the trial.

Align the model with system and user goals

Illustrative energy forecasting: a better score helps only through a useful decision.

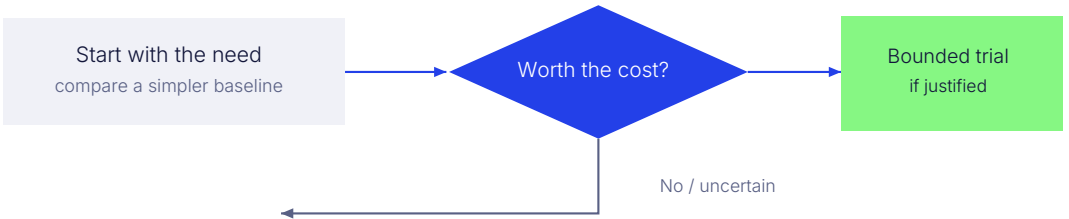


Scope: decision + success evidence + milestones + owner + time / cost budget

A model metric is one piece of success evidence; measure the outcome it is meant to support.

Choose ML only when the evidence supports it

Compare rules, human review and existing tools before adding a learned component.



Retain the simpler option or collect evidence.

Assess data, error costs, review or fallback, explanations, reuse and lifetime effort.

Prediction error is uncertain. This does not mean every ML prediction is computationally random.

References: evaluation and persistence

Reference reading: Primary documentation and original research.

1 Leakage and validation

scikit-learn.org/stable/common_pitfalls.html
scikit-learn.org/stable/modules/cross_validation.html

2 Metrics and performance

scikit-learn.org/stable/modules/model_evaluation.html
scikit-learn.org/stable/computing/computational_performance.html

3 Model persistence and temporal folds

scikit-learn.org/stable/model_persistence.html
scikit-learn.org: TimeSeriesSplit API reference

Documentation interfaces evolve; the concepts and the stated evaluation boundary govern the examples.

References: lineage and gate mechanics

Reference reading: Primary documentation and original research.

1 Tracking, registry and model signatures

mlflow.org/docs/latest/ml/tracking/
mlflow.org/docs/latest/ml/model-registry/workflow/

2 Interfaces and replay

mlflow.org/docs/latest/ml/model/signatures/
airflow.apache.org/docs/apache-airflow/stable/best-practices.html

3 CI outcomes and finite-number checks

docs.github.com: Troubleshooting required status checks
docs.python.org/3/library/math.html#math.isfinite

Documentation interfaces evolve; the concepts and the stated evaluation boundary govern the examples.

References: compute and parallel training

Reference reading: Primary documentation and original research.

1 Compute estimation and CPU resources

epoch.ai/publications/estimating-training-compute
scikit-learn.org/stable/computing/parallelism.html

2 Distributed and pipeline parallelism

docs.pytorch.org: Distributed Data Parallel notes
docs.pytorch.org: Pipeline Parallelism and tensor parallel API

3 Worker orchestration

docs.ray.io/en/latest/train/overview.html
Ray Train: training function, workers, scaling configuration

Documentation interfaces evolve; the concepts and the stated evaluation boundary govern the examples.

References: compression and serving

Reference reading: Primary documentation and original research.

1 Pruning and quantization

tensorflow.org/model_optimization/guide/pruning
onnxruntime.ai/docs/performance/quantization

2 Distillation and ONNX

Hinton, Vinyals & Dean (2015), arxiv.org/abs/1503.02531
Sanh et al. (2019), arxiv.org/abs/1910.01108; onnx.ai

3 Serving engines and maintenance

docs.nvidia.com: Triton architecture/batchers; Dynamo
docs.vllm.ai/en/latest/, docs.pytorch.org/serve/

Documentation interfaces evolve; the concepts and the stated evaluation boundary govern the examples.

References: platforms and uncertainty

Reference reading: Primary documentation and original research.

1 Components and caching

kubeflow.org: Pipelines: Component; Use Caching
docs.aws.amazon.com/sagemaker/latest/dg/pipelines.html

2 Managed pipeline responsibilities

learn.microsoft.com: Azure Machine Learning pipelines
docs.cloud.google.com: ML pipeline introduction

3 Time dependence and practical ML value

Künsch (1989), doi.org/10.1214/aos/1176347265
bashtage.github.io/arch: time-series bootstraps; Google Rules of ML

Documentation interfaces evolve; the concepts and the stated evaluation boundary govern the examples.